#1 Method `opticsCluster` takes the ordering and reachability distances produced by `optics`, and a parameter `eps<=epsMax`, used to extract a flat clustering from the (infinitely many) possible clustering computed by OPTICS. It returns an array of the cluster indices associated with the points (or, equivalently, a dictionary associating points to cluster indices).
#2 Initialize the index of current cluster.
#3 Initialize a flag that is used to keep track of when a previous cluster ended.
#4 Initialize the indices of the cluster assignments: initially all points are marked as outliers.
#5 Cycle through all points in the order they were processed.
#6 Check if the reachability distance of a point is undefined or larger than the core radius epsilon.
#7 If so then we need to close current cluster.
#8 Otherwise, check if it needs to start a new cluster (by incrementing current index)
#9 Actually perform the increment.
#10 If `incrementClusterIndex` is true, it means that we have found the first point in the new cluster with a non-null reachability distance; this means that current point is reachable from the previous point in the ordering (which, because of the way OPTICS work, will certainly have a reachability distance that's `null` or too large). So, we need to include previous point as well in current cluster.
#11 Add the `i`-th point in the ordering to current cluster.
#12 Since we added current point to a cluster, we know that we need to check if the next points are also reachable from it.
#13 Return a flat clustering, an array with the cluster index of each point.